

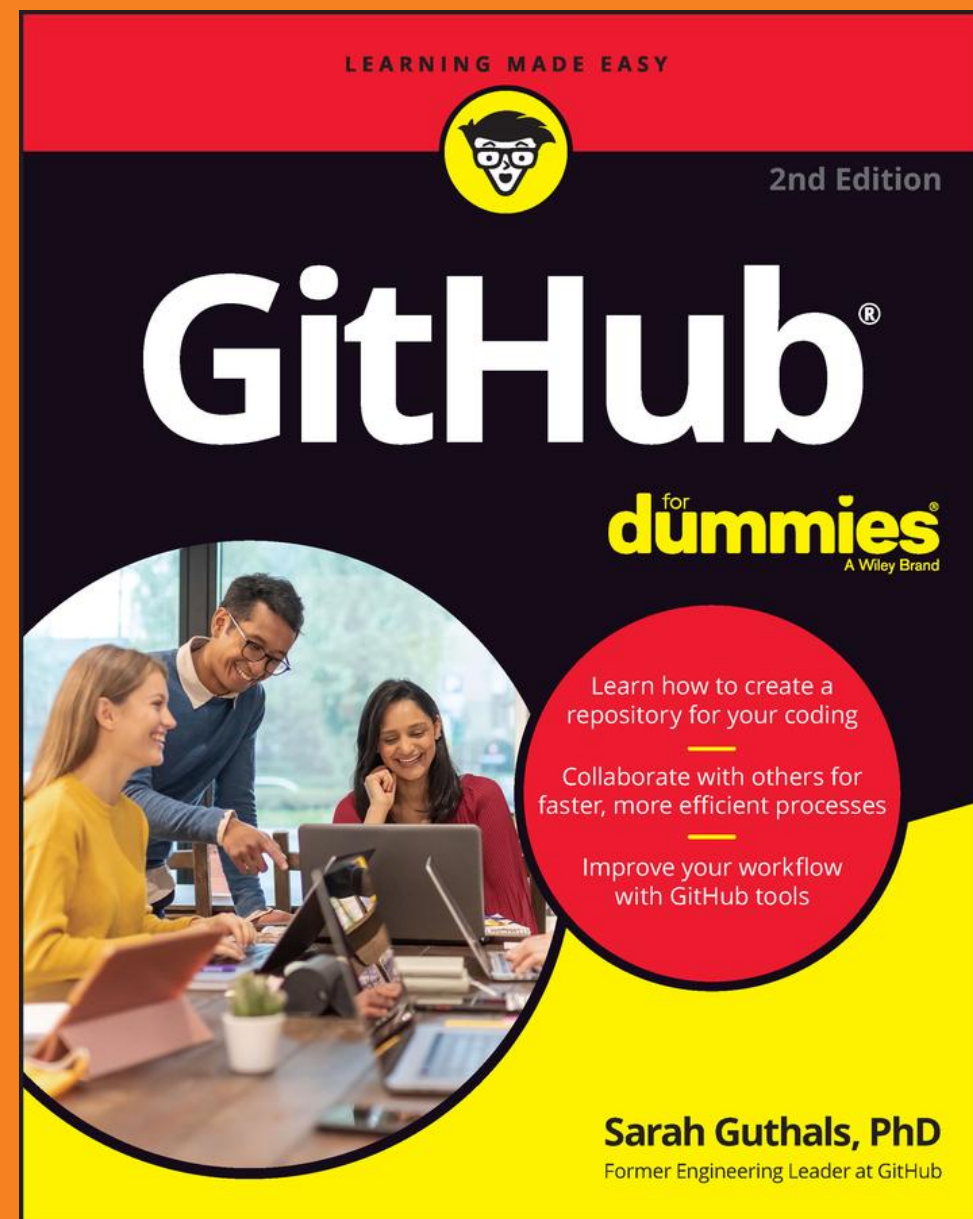
Source Code Management

Explore effective strategies and tools for optimizing source code management in software development.



CSV 202

L:0 T:0 P:2 Credits:1



► Textbooks:

GITHUB FOR DUMMIES by PHIL
HAACK SARAH GUTHALS, WILEY

► References:

1. EFFECTIVE DEVOPS: BUILDING A
CULTURE OF COLLABORATION,
AFFINITY, AND TOOLING AT SCALE by
JENNIFER DAVIS, O'REILLY

Vision

To be a globally recognized school through excellence in teaching, learning and research for creating Computer Science professionals, leaders and entrepreneurs of future contributing to society and industry for sustainable growth.

Mission

- To build computational skills through hands-on and practice-based learning with measurable outcomes.
- To establish a strong connect with industry for in-demand technology driven curriculum.
- To build the infrastructure for meaningful research around societal problems.
- To nurture future leaders through research-infused education and lifelong learning.
- To create smart and ethical professionals and entrepreneurs who are recognized globally

Course Outcomes

CO1 :: Illustrate the installation and configuration of Git

CO2 :: Identify the differences in various commits and working with Git branches

CO3 :: Understand the use of Rollbacks in Git

CO4 :: Analyze how to resolve Merge conflicts and identifying the usecase of rebase and revert

CO5 :: Demonstrate the working of GitHub

CO6 :: Understand the mechanism of cloning of GitHub repository

Skills To Be Learned

- Learn the importance of SCM in software development, including how it supports collaborative coding and ensures code integrity.
- Gain knowledge about popular SCM tools like Git, Subversion, Mercurial, and Perforce, and understand their key features and advantages.
- Learn key practices for effective source code management, such as regular commits, clear commit messages, and code reviews to maintain high standards in code quality and functionality.

Software Requirements



Certification/MOOC



- Certification: AWS CERTIFIED DEVOPS ENGINEER – PROFESSIONAL
- Platform: AMAZON WEB SERVICES
- [URL:https://aws.amazon.com/certification/certified-devops-engineer-professional/](https://aws.amazon.com/certification/certified-devops-engineer-professional/)
- Benefits: 1 CA

CSV 202: Source Code Management Laboratory

CA category A0303

Exam Category X6 (Att. 5, CAP 45, ETT 50)

Syllabus

List of Practicals / Experiments:

Introducing Version Control System – Installing Git CLI on Windows, Mac & Linux

- Explain the Git Architecture & install Git on whatsoever OS participants are using. Ensure git commands are working & then configure Git.

Setting up a Git Repository & Initialization

- Explain the Repository Initialization & all the hidden artifacts involved in the process. Perform Reinitialization practical as well.

Working with Git – File States, Project Section, Workflow

- Perform the practical of tracking files through git, working with staging area, working with commit area. Different methods involved in working with commit & staging area. Explain commit id & tracking process of Git.

Working with Git History

- Work with git logs & its different variations to obtain different insights. Mention the importance of history in git.

Identifying Binary Files, Diffing Binary Files

- Explain Binary Files. Also perform the practical of getting the difference between various commits. Work with git diff command w.r.t various stages in git workflow.

Working with git branches

- Explain branches in git., explain all the ways using which different branches can be created. Explain the branch merging & its need as well.

Rollbacks in Git

- Explain the significance of HEAD, show the practicals for performing a rollback in timeline of git. Explain different types of resets in git.

Merge Conflicts Resolution in Git

- Explain Merge Conflicts, its source, & its solution practically with different strategies.

Rebasing & revert in Git

- Explain rebase, perform its practical, explain the difference between a merge & a rebase. Perform the practical for revert as well.

Introducing GitHub & Exploring GitHub

- Create a GitHub account & provide a tour of GitHub to the audience. Explain various elements of GitHub. Create a Repository on GitHub of public as well as private mode.

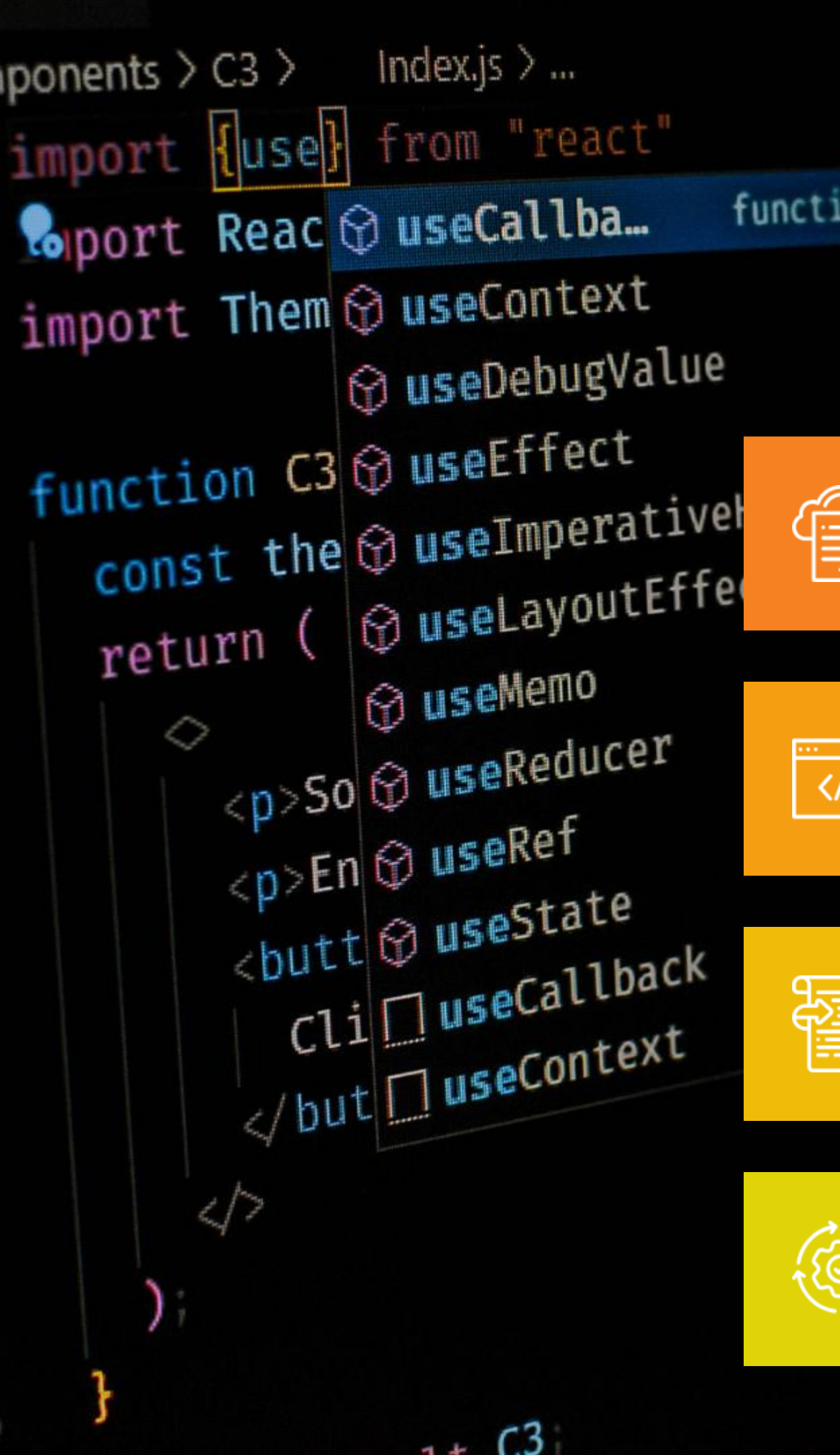
Cloning an existing Repository & remote setup

- Clone a GitHub Repository. Create a local Git repo & attach it to GitHub. Perform the complete remote setup.

Introduction to Source Code Management

Exploring the Essentials of SCM for Developers

01 Importance of SCM Understanding the role of SCM in development.	02 SCM Tools Overview of popular tools used in SCM.	03 Best Practices in SCM Key practices for effective source code management.	04 Version Control Systems Exploring version control systems like Git.
05 Collaborative Development How SCM supports collaborative coding.	06 Maintaining Code Integrity Ensuring code integrity through SCM.	07 Future Trends in SCM Emerging trends and technologies in SCM.	



Key Concepts of Source Code Management

Understanding Repositories, Branches, Commits, and Merges



Repositories

Storage locations for code.



Branches

Allow developers to work on different features simultaneously.



Commits

Snapshots of code changes.



Merges

Integrate changes from different branches.

Understanding Version Control Systems

Key Features and Types of VCS

 Definit on of VCS manage changes to source code over time.	 Purpos e of They provide versioning, branching, and merging.	 Mechan isms of Key features include tracking modifications.	 Types of VCS Centralized and distributed systems exist.
 Advant ages of Centrali zed VCS Easier management but single point of failure.	 Advant ages of Distribu ted VCS Better fault tolerance and offline capabilities.	 Importa nce in VCS are fundamental to efficient software control.	

Git Workflow Steps

Key Steps for Effective Collaboration

01 Clone a Repository

Create a local copy for changes.

02 Create Feature Branches

Develop new features separately.

03 Make Commits

Record changes with messages.

04 Merge Branches

Integrate feature branches into main.

05 Push Updates

Upload local changes to remote.

Branching and Merging Strategies

Understanding strategies for efficient code management

01 Feature Branching

Utilizes separate branches for new features.

02 Git Flow

A structured branching model for releases.

03 Trunk-based Development

Continuous integration with a single branch.

04 Choosing the Right Strategy

Critical for team collaboration and efficiency.

05 Managing Code Changes

Essential for minimizing conflicts and errors.

06 Minimizing Integration Issues

Reduces risks of integration failures.

Key Features of GitHub

An Overview of GitHub's Capabilities

01

Popular platform for hosting Git repositories

GitHub is widely used for managing code projects.

02

Facilitates collaboration among developers

Supports teamwork through pull requests and reviews.

03

Integration with numerous tools

Offers compatibility with various development tools.

04

Support for open-source and private projects

Enables both open-source contributions and private work.

05

Vibrant community as a valuable resource

Active user community provides support and resources.

Best Practices in SCM

Ensuring Efficient and Reliable Code Management



Importance of Code Reviews in SCM

Enhancing Code Quality and Team Productivity

Vital Part of SCM Process

Code reviews are essential for effective SCM.

Promotes Code Quality

Ensures that code meets quality standards.

Facilitates Knowledge Sharing

Encourages team members to learn from each other.

Peer Reviews Before Integration

Changes are reviewed by peers prior to merging.

Provides Constructive Feedback

Helps developers understand and improve coding practices.

Ensures Adherence to Standards

Checks compliance with established coding standards.

Enhances Team Productivity

Streamlined processes lead to more efficient teamwork.

Effective Collaboration Tools

Utilization of tools that facilitate code reviews.

Techniques for Managing Repositories

Addressing Challenges in Performance and Merges



Challenges of Large Repositories

Large repositories can lead to slow performance.



Leveraging Large File Storage

LFS can improve handling of large files within repos.



Merging Complexities

Merging code in large repositories can become difficult.



Repository Splitting

Consider splitting repositories to enhance performance.



Using Submodules

Submodules can help manage dependencies effectively.

Backup and Recovery Strategies

Essential Approaches to Protect Codebase

Regular backups

Scheduled backups to safeguard codebase integrity.

Data replication

Ensures availability by duplicating data across locations.

Disaster recovery plans

Structured response plans for unexpected data loss.

Quick restoration

Swift recovery processes in case of hardware failures.

Maintaining project continuity

Ensures ongoing project success despite disruptions.

Best Practices in SCM Implementation

Key Insights and Lessons Learned

Effective Use of SCM Tools

Utilized SCM tools for process efficiency.

Streamlined Development Processes

Improved workflows to reduce delays.

Enhanced Collaboration

Facilitated better team communication.

Improved Code Quality

Incorporated best coding practices.

Successful Project Outcomes

Achieved goals through effective SCM.

Challenges in SCM

Identifying Key Issues in SCM Practices

Managing complex branching

Navigating multiple branches can lead to confusion.

Handling large codebases

Large repositories require efficient organization.

Ensuring security

Protecting code from unauthorized access is crucial.

Maintaining efficient collaboration

Teamwork is essential for successful SCM.

Addressing Supply Chain Management Challenges

Key Strategies for Success in SCM

Strategic Planning is Essential

Effective SCM requires well-thought-out strategies.

Adopting Agile Methodologies

Flexibility in processes enhances response time.

Leveraging Automation

Automation streamlines operations and reduces errors.

Implementing Robust Security Measures

Security protects data and builds trust in SCM.

Continuous Process Refinement

Regular updates improve efficiency and adaptability.

Benefits of Open Source SCM Tools

Key Highlights of Open Source Tools

01 Flexibility of Tools

Tools like Git and Mercurial allow customization.

02 Community-Driven Development

Development driven by active community contributions.

03 Cost-Effective Solutions

Open source tools provide low-cost code management.

04 Active Community Contributions

Communities enhance tools to keep them relevant.
